A Qwen–CLIPS Neuro-Symbolic Architecture for Plan Generation in Service Robotics: Design, Implementation, and Comparative Evaluation with Direct GPT

Keywords: Service Robotics, Neuro-Symbolic Planning, Large Language Models, CLIPS, QLoRA, RoboCup@Home, GPSR.

Abstract: Transforming natural-language commands into robot plans requires distinguishing semantic interpretation, planning, and executor compatibility. We present Qwen–CLIPS with Qwen2.5-3B and Qwen2.5-7B interpreters fine-tuned using QLoRA on 40,000 examples, compared with direct GPT using structured output. The exploratory offline evaluation, bounded by the implemented coverage of the General Purpose Service Robot (GPSR) task in the RoboCup@Home league and contrasting cases, comprises 60 English commands. For the 46 commands within the local generator's scope, symbolic fulfillment was 220/230, 215/230, and 216/230 runs for 3B, 7B, and GPT, respectively; scenario exceptions are kept separate. Qwen retained its goals and plans across runs, including failures. GPT exhibited literal plan changes for 24/60 commands and changes in an operational-sequence projection for 9/60; these differences do not automatically imply errors. Incompatible arguments, omissions, and redundant steps were identified despite structural validity. The results characterize stability, coverage, and observable resources on a development set without measuring physical execution.

1 INTRODUCTION

[1] Service robots must transform human instructions into actions compatible with their navigation, perception, manipulation, and interaction modules. This requires interpreting goals, entities, and relations and producing a plan that respects the executor's repertoire and parameters. Language models provide linguistic flexibility (Vaswani et al., 2017; Shi et al., 2023); explicit rules support inspection of symbolic expansion (NASA Johnson Space Center, 1991; Giarratano and Riley, 2005).

We focus on *General Purpose Service Robot* (GPSR), a task in RoboCup@Home, a competition for service robots in domestic environments. GPSR presents spoken commands combining tasks such as finding people or objects, transporting objects, and interacting with users. We delimit coverage through the families, structures, and vocabulary of a locally extended generator and the actions available in the service robot. This scope does not represent all possible GPSR commands or universal robotic planning. We evaluate fulfillment within the intended coverage, linguistic variation, and requests exceeding its boundaries.

We compare Qwen–CLIPS, with fine-tuned Qwen2.5-3B and Qwen2.5-7B interpreters, against direct GPT. The former separates neural interpretation from rule-based expansion; the latter generates goals and a plan in one response. We ask how these configurations differ in fulfillment, consistency, linguistic robustness, and observable cost for the specified tasks. Our initial hypothesis anticipates greater structural validity, traceability, and consistency from Qwen–CLIPS and greater flexibility from GPT for novel formulations; we examine these dimensions separately.

We contribute an intermediate goal representation, its connection to CLIPS and a service-robot executor, two QLoRA-adapted interpreters, and a reproducible comparison of three configurations on 60 commands with five runs per system. Evaluation ends at the symbolic plan and retains failures, clarifications, and rejections.

2 RELATED WORK

[2] CLIPS represents knowledge through facts and forward-chaining production rules (NASA Johnson Space Center, 1991; Giarratano and Riley, 2005). Its behavior depends on an explicit rule and fact base,

[1] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

[2] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

which supports inspection while delimiting coverage. Language models can relate instructions to symbolic representations or actions (Shi et al., 2023); constraining their output format through a schema does not itself establish semantic correctness.

Neuro-symbolic architectures combine neural learning with explicit knowledge representation (d'Avila Garcez et al., 2015; Besold et al., 2017). LLM+P (Liu et al., 2023) translates a description into PDDL (*Planning Domain Definition Language*), a language for specifying planning domains and problems, and uses a classical planner to search for a plan. Our separation of responsibilities is related, but uses canonical goals and production-rule expansion rather than PDDL search. Comparison with direct GPT examines the consequences of this separation under a shared repertoire.

LoRA adapts low-rank matrices while freezing original weights (Hu et al., 2022); QLoRA combines adapters and quantization to reduce memory (Dettmers et al., 2023). These techniques support specializing Qwen2.5 interpreters (Yang et al., 2024) for a bounded domain and comparing two sizes with a common corpus.

3 PROBLEM FORMALIZATION

[3] We represent the transformation as $x \to G \to P$, where x is the command, $G = (g_1, \ldots, g_n)$ the goals, and $P = (a_1, \ldots, a_m)$ the actions. The canonical vocabulary comprises `go`, `find`, `take`, `deliver`, `place`, `guide`, `follow`, `count`, `tell`, `save`, `answer_question`, and `greet`. For `find` and `count`, `kind` distinguishes people from objects; qualifiers express attributes such as gesture, posture, or clothing.

Goals retain precedence, reference, and state dependencies. For example, taking an object precedes delivering it, and navigating to a room establishes the spatial context for a search. We distinguish syntactic and structural validity, semantic fidelity of G, fulfillment by P, and physical success; the last is outside the measurement scope.

4 METHODOLOGY

[4]

[3] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

[4] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

4.1 Platform and Domain

The experimental action contract follows a service-robot platform described in prior work (Savage et al., 2024). Its navigation, perception, manipulation, and interaction modules are organized through the ViRbot control architecture, see figure 1, and communicate through ROS 2, software infrastructure for connecting robotic components, figure 2. They provide the executor contract: actions, admitted arguments, and conventions that plans must respect. We take GPSR (Hart et al., 2026) as a reference and use a locally extended `CommandGenerator` (RoboCup@Home, 2026a), organized around `CompetitionTemplate` (RoboCup@Home, 2026b). Catalogues delimit people, objects, categories, rooms, and locations; additional templates broaden the formulations of supported tasks.

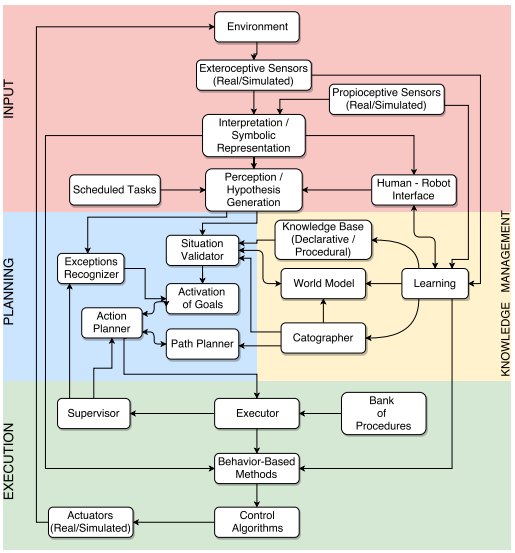


Figure 1: ViRbot system to operate service robots.

4.2 Fine-Tuning Corpus

We construct command–goal pairs through shared context between linguistic and semantic generation, preserving entities and dependencies across subtasks. A family groups related task structures; a template specifies a way of expressing them with variable entities. Both models use the same `dataset_gpsr.jsonl`: 40,000 records, 29 families, 18 goal signatures, and 87 template identifiers. A random row-level split allocates 36,000 training and 4,000 validation examples with seed 42. All labels passed schema validation; the first 200 valid labels passed the CLIPS expansion check. This is a prefix check rather than a random corpus sample.

4.3 Generation Paths

Figure 3 compares the paths. Qwen produces G in JSON; a deterministic module validates and converts goals into `gpsr-goal` facts, and CLIPS expands them into P while maintaining expected state. GPT receives the command, catalogues, repertoire, constraints, and examples, producing (G,P) or clarification/rejection through structured output (OpenAI, 2026b). Auxiliary control supplies reference goals directly to CLIPS to locate interpretation, adaptation, or expansion errors.

5 IMPLEMENTATION

5

5.1 Environment and Training

The implementation is integrated into the ViRbot architecture of the service robot.

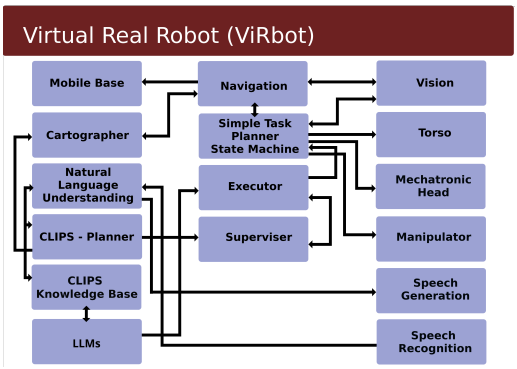


Figure 2: ViRbot system implemented using ROS-2.

Communication with the platform is done through ROS 2 (Jazzy). Training was performed on a platform with an NVIDIA GeForce RTX 4070 SUPER GPU (12 GB), an 11th-generation Intel Core i7 processor, and 32 GB of RAM. The recorded environment includes Ubuntu 24.04.5 LTS, Python 3.12.3, PyTorch 2.9.0+cu130, Transformers 4.51.3, PEFT 0.18.1, bitsandbytes 0.49.2, and clipspy 1.0.5.

We fine-tune both interpreters with QLoRA using 4-bit NF4, double quantization, rank 32, scale 64, and dropout 0.05. Loss is computed on response tokens. We retain an effective batch size of eight, increasing the per-device batch for 3B. Table 1 summarizes the differences, and Figure 4 shows the complete logs. Our retrospective duration estimate places 3B fine-tuning at approximately one third of 7B's time; this is not an instrumented measurement of training cost.

Table 1: Configuration and checkpoint selection.

Parameter	Qwen3B	Qwen7B
Per-device batch	4	1
Accumulation	2	8
Epochs	2	2
Updates	9,000	9,000
Selected step	7,000	5,000
Validation loss	0.22720	0.23214

For 7B we use maximum length 512, AdamW, learning rate 10^{-4} with cosine scheduling, 150 warm-up steps, and BF16. A generative check of its checkpoint on 50 validation examples obtained 50 exact matches, all schema-valid and accepted by CLIPS. This check is distinct from the 60-command experimental set.

5.2 Inference and Symbolic Contract

We normalize Qwen inputs through lowercasing, whitespace, punctuation, politeness expressions, and aliases. We load the quantized model and adapter and use `do_sample=False`, repetition penalty 1.05, and at most 128 new tokens. The adapter preserves step, goal, entity, type, destination, relation, and qualifiers. CLIPS processes goals in order and updates location and held object as expected effects rather than physical observations. Figure 5 illustrates an expansion.

For GPT we use `responses.create`, model identifier `gpt-5.6-terra`, a customized prompt, version 1.6, and at most 8,192 output tokens. The strict schema requires `status`, `message`, `goals`, `plan`, `start_note`, and `end_note`. The validator checks structure, status coherence, numbering, parameter count, and closure; semantic compatibility with the executor is assessed separately. ROS 2 integration connects the semantic node, adapter, and CLIPS node; campaigns use independent programs that record the plan without sending it to controllers.

6 EXPERIMENTAL EVALUATION AND RESULTS

6

[5]AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

[6]AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

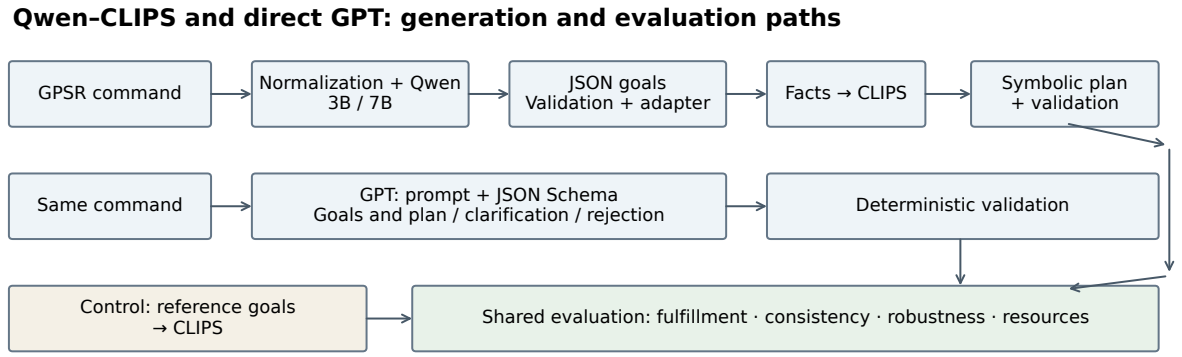


Figure 3: Experimental paths and auxiliary control. Outputs are assessed under shared criteria without physical execution.

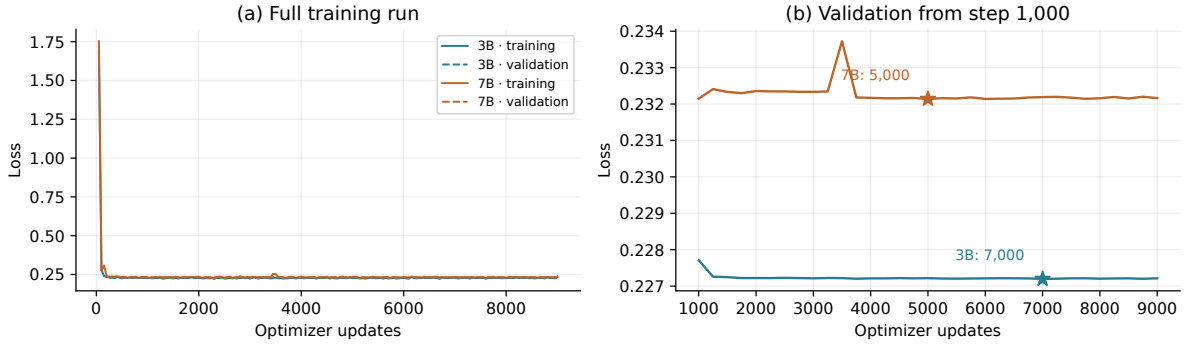


Figure 4: Fine-tuning of 3B and 7B from original logs: training and validation recorded every 50 and 250 steps, respectively. Stars mark the selected checkpoints.

6.1 Design and Provenance

We evaluated 60 English commands in five runs per configuration across three systems: 900 first attempts. R1 is the initial campaign; R2–R5 are four additional repetitions. Errors and non-executable responses were retained without replacement by retries. Hashes for inputs, references, rules, adapters, and Qwen model configuration agree across phases; GPT retains its prompt, schema, and explicit parameters. Additional GPT requests include a two-second pause outside individual latency. The same remote model identifier does not certify identical provider weights across dates. Qwen campaigns query the local HTTP server; GPT campaigns query the remote API.

An intent identifies a task with its entities and constraints, shared by different linguistic formulations. The set contains 28 intents: 16 with three formulations (canonical, paraphrase, and linguistic noise), totaling 48 commands, and 12 diagnostic cases. Formulations and repetitions of an intent are not independent observations. This development set was used during protocol and prompt refinement. The audit identified eight normalized corpus matches across four intent groups, although no literal matches; neither semantic independence nor absence of GPT training exposure is claimed. Reviewed experimental case sheets establish subtasks, constraints, and reference goals. We expand the 54 references with executable tasks through CLIPS as an auxiliary diagnostic, not additional performance measurements or infallible ground truth. The remaining six cases concern ambiguity, unavailable capabilities, or contradiction and are assessed for response adequacy.

6.2 Coverage Classification

Table 2 presents the blocks approved following the audit. We established the classification after the first run as an exploratory analysis of the development set. Generator membership depends on structure and entities, not on model correctness. Paraphrase and noise constitute an additional axis within each block.

G comprises 18 intents. L contains three formulations with `table`, a location not literally in the catalogue, although the placement structure is supported. C contains three versions of two consecutive navigation goals and one command with two complete deliv-

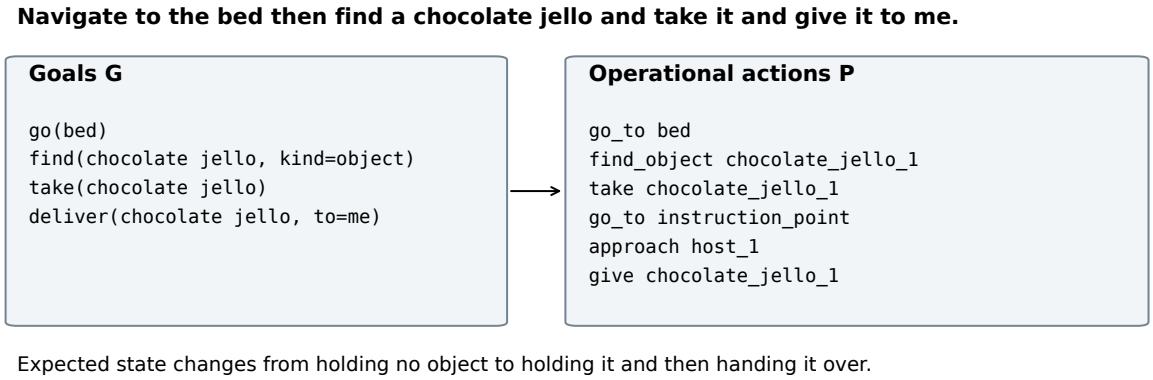


Figure 5: Example goals and symbolic expansion. Announcements and confirmations are omitted to show the main actions; recorded plans retain them.

Table 2: Distribution relative to the locally extended generator.

Block	Criterion	*n*
G	Supported structure and entities	46
L	Location vocabulary extension	3
C	Compositions outside local generation paths	4
M	Unsupported attribute conjunction	1
A	Missing required information	3
F	Actions outside the repertoire	2
X	Contradiction	1
	Total	60

eries. M requires simultaneous gesture and clothing attributes. A, F, and X evaluate clarification, recognition of limitations, and contradiction; inventing an executable plan is not required for these inputs.

6.3 Evaluation Criteria

We use symbolic fulfillment as the primary criterion: plans must preserve subtasks and constraints, use executor-compatible actions and arguments, and respect necessary dependencies. Valid JSON does not guarantee satisfaction of the command.

Fulfills indicates task satisfaction; *fails* indicates an observable omission, incompatibility, or violation. *Not verifiable* indicates insufficient evidence to resolve an identity or protocol dependency; such cases would remain in the denominator of evaluable tasks, separate from successes and confirmed failures. No cases remain in this category after clarifying the scenario conditions. *Not applicable* means the case specification does not require an executable plan: in blocks A/F/X, we separately assess the adequacy of clarification or rejection. We do not assign this judgment merely because the system fails or produces no plan. An observable defect determines failure even when other effects cannot be verified.

6.4 Scenario Conditions and Review Procedure

In named-recipient tasks, the only person in the room represents the requested person. For following tasks, the first person found at the specified location is the intended recipient, and `instruct_follow_to_*` activates the executor special case that carries out following and its additional steps. These conditions do not extend to counting or GPSR in general and do not imply physical verification. Interaction and memory protocols retain the scope documented in the review.

We transfer previous judgments only to identical evidence. For the 70 GPT runs differing from R1 (42 variants across 28 commands), we use AI-based review against requirements and executor code, kept separate from our explicit acceptance decisions. Subsequent confirmation of the following conditions is recorded separately and does not extend to the remaining automated judgments. We consider responses without plans equivalent when they retain an adequate request for information or explanation of a limitation or contradiction. Each record retains evidence, judgment provenance, and exceptions.

6.5 Symbolic Fulfillment

Table 3 preserves the original constraints. Incomplete capabilities were identified for clothing-filtered counting and weight-based selection. Exceptions concern unfiltered clothing-related counts and the use of size as a proxy for weight; they remain classified as failures under the original criterion. In G, GPT fulfills 40/46 commands in all five runs; four commands

Table 3: Block G: 46 commands and 230 runs per system. Exceptions are a subset of failures, not an additional category.

System	Fulfills	Fails	Exceptions	Fulfills 5/5
Qwen2.5-3B–CLIPS	220	10	10	44/46
Qwen2.5-7B–CLIPS	215	15	10	43/46
GPT	216	14	8	40/46

change verdict across runs, including correction of the missing following trigger in R2–R5; R1 retains that failure.

6.6 Consistency, Arguments, and Additional Steps

Table 4 summarizes differences across five runs of each command. Both Qwen systems retain goals, plans, and failure patterns, consistent with generation without sampling. The operational projection removes numbering and only `say` announcements beginning with "I will" or "I have"; it retains other actions, arguments, and dialogue keys. This descriptive measure does not prove functional equivalence: it includes redundancy and capitalization changes. We assess fulfillment stability separately.

Table 4: Commands with differences across their five runs, out of 60 per system. Failure patterns are included; stability does not imply fulfillment.

System	Goals	Literal plan	Operational projection
Qwen3B–CLIPS	0/60	0/60	0/60
Qwen7B–CLIPS	0/60	0/60	0/60
GPT	4/60	24/60	9/60

Inspection distinguishes spoken text from symbolic identifiers. Location names containing spaces within descriptive announcements do not imply invalid navigation arguments. No navigation arguments containing spaces were found in these GPT responses. The `focus` action enables or disables visual attention during interaction; some `say` keys also trigger protocols in addition to speech. One run contains four `focus` steps with arguments such as "focus enable" instead of "enable". The current parser compares exact strings and does not enable attention with the former. Two other runs request weight-based selection through an argument unsupported for that operation, which the code interprets as a class for counting. These are three runs across two commands with incompatibility identified by inspection despite passing the recorded structural validator, not observed physical failures.

Additional steps are classified by their effects. Two runs contain redundant attention resets and one repeats the name query; these preserve subtasks but alter protocol or efficiency. Another run omits an explicit search goal while preserving it in the plan. One also omits navigation to the recipient's room and fails the task. Four add the executor-recognized following trigger, correcting its initial omission; they fulfill the task under the scenario identity and protocol conditions. All five runs of each Qwen system already contained that trigger and also fulfill the task. Neither every added step is counted as an error nor every change as a correction.

6.7 Coverage and Linguistic Robustness

In the other blocks of Table 2, fulfillment in C is 15/20, 5/20, and 20/20 runs for 3B, 7B, and GPT; in L, each achieves 15/15. The small composition block does not isolate length effects. In M, GPT changes from prudent rejection in R1 to four plans omitting a clothing condition; Qwen also loses a condition. In A/F/X, GPT retains adequate responses without plans in 30/30 runs despite wording changes, versus 0/30 for Qwen under the configured contract. This difference is not interpreted as isolated intrinsic model capability.

Linguistic robustness is examined through 16 formulation triplets, 14 within G, preserving intent-level pairing. Across the 14 G triplets, all three formulations fulfill jointly in 70/70 intent–repetition combinations for 3B, 65/70 for 7B, and 67/70 for GPT. This is a descriptive summary of 14 intents, not 70 independent samples. Repetitions characterize stability for identical input rather than adding linguistic diversity. Results are retained per formulation and run without selecting the best attempt. Overall, 3B produced 290 plans and ten errors; 7B, 280 plans and twenty errors; GPT, 269 plans, fifteen clarifications, and sixteen rejections, without program execution errors. Qwen errors are not counted as correct rejections.

6.8 Latency, Resources, and Cost

Table 5 summarizes observed latency. R1 and R2–R5 are also shown separately in Figure 7, given the added GPT pause and temporal conditions of sequential campaigns. Differences are not causally attributed

Table 5: Total latency in G, 230 runs per system. Q25–Q75 contains the central 50 % of response times. Descriptive quartiles without assuming independence.

System	Mean (s)	Median (s)	Q25–Q75 (s)
Qwen3B	2.47	2.50	2.08–2.79
Qwen7B	1.99	2.03	1.68–2.24
GPT	4.55	4.37	3.40–5.32

to model size. Median sampled GPU allocator peaks remain approximately 2.15 GiB for 3B and 5.50 GiB for 7B; these do not represent energy, total card memory, or memory exclusive to one command. We report GPT's internal resources as *unavailable*, without substituting client consumption.

The 300 GPT requests recorded 1,642,345 input and 141,923 output tokens. We recorded US$0.42 for the first 60 requests and US$1.65 for the additional 240 from the Usage dashboard: US$2.07 overall and US$0.0069 per request. These amounts cover the campaigns and were retrieved from the OpenAI Usage dashboard (OpenAI, 2026c); they exclude development and total ownership cost. We do not assign zero cost to local computation.

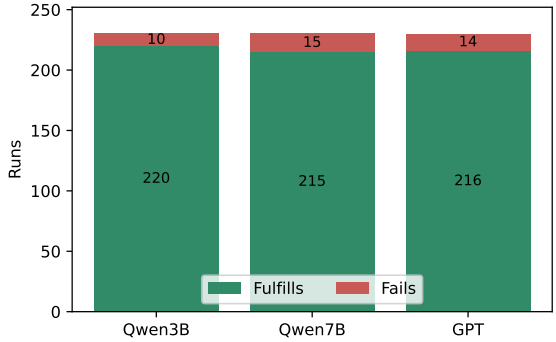


Figure 6: Fulfillment in G: 230 runs per system. Exceptions remain classified as failures.

7 DISCUSSION

7

7.1 Interpretation of the Results

The larger interpreter did not yield greater fulfillment: 3B achieved more correct runs than 7B in the main block, whereas 7B showed lower latency and higher GPU memory use. These observations describe the measured configurations rather than a causal relationship between size, quality, and speed. The Qwen–CLIPS separation helps locate whether a constraint is lost in goals, adaptation, or expansion.

Repeatability must be interpreted alongside fulfillment. Without sampling, Qwen retains both successes and failures; repeating an incorrect output does not make it more reliable for the task. GPT exhibits wording and sequence variation, including corrections and new omissions. Its clarification and rejection behavior is adequate in the diagnostic cases but changes for the attribute conjunction. We therefore assess decision stability and coverage alongside literal equality.

Incompatible arguments show that JSON structure and parameter count do not guarantee a usable plan. Verification must cover admitted values, references, and action effects. Our initial hypothesis receives descriptive support for Qwen consistency, but these data do not establish its general superiority in structural validity. Traceability is a property of decomposition and records rather than a measured comparative rate; linguistic robustness is limited to the included formulations.

7.2 Scope of the Evidence

Fine-tuning data were split by rows rather than by family or template, and the experimental set informed protocol development. Results do not establish generalization to unseen families. Five repetitions and paraphrases of an intent remain related; we present descriptive statistics without treating them as 900 independent samples. Small diagnostic blocks, retrospective classification, and AI-based review delimit the strength of the conclusions. Simultaneous changes in model, adaptation, and planner prevent attributing all differences to CLIPS.

For deployment, Qwen–CLIPS offers control of weights, rules, and versions and local inference without external connectivity once installed, at the cost of hardware, energy, and maintenance. GPT externalizes part of that infrastructure through an API requiring connectivity and usage expenditure. Selection depends on coverage and operational conditions; this evaluation does not calculate comparative total ownership cost.

7.3 Future Work

The next evaluation will use independent commands with predefined blocks and criteria. We propose extending argument validation, addressing filtered counting and weight-based selection, exploring clarification in Qwen–CLIPS, and testing plans through physical execution. Additional families and lan-

[7] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

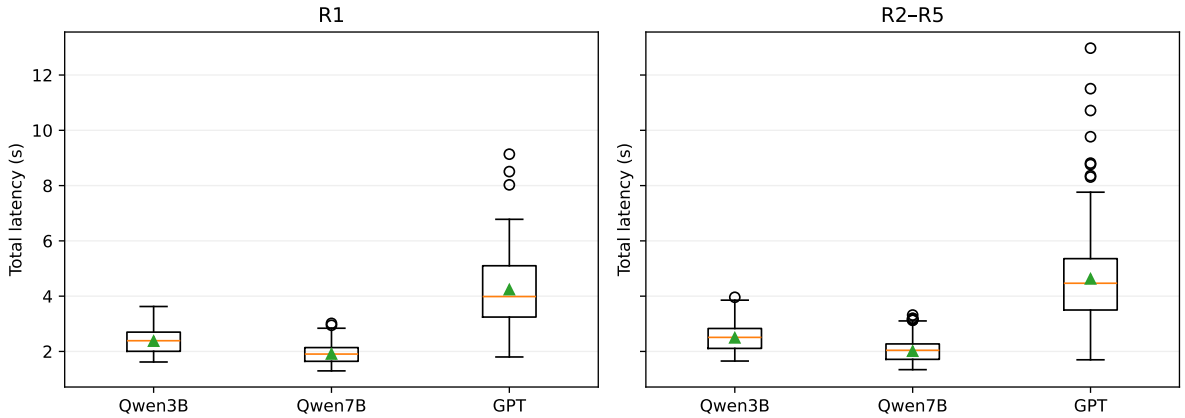


Figure 7: Total latency distribution in G by phase. Boxes: Q25–Q75; line: median; triangle: mean; whiskers extend to 1.5 interquartile ranges with outliers shown. Inter-request pauses are excluded from individual latency.

guages will support studying transfer beyond current coverage.

8 CONCLUSIONS

[8] We compared three configurations for transforming GPSR commands into symbolic plans under a common contract. Qwen–CLIPS retained stable outputs and supported inspection of intermediate stages; GPT combined adequate responses to domain limits with variations including incompatible arguments and omissions. The main finding is that fulfillment, repeatability, and valid format describe different properties and should be measured separately. The data support a bounded evaluation of these configurations and guide extension of the contract and coverage without establishing a universal winner.

USE OF AI TOOLS

OpenAI ChatGPT and Codex were used to assist with drafting, translating, and editing Sections 1–8, preparing LaTeX and analysis code, and creating the programmatic diagrams and data plots in Figures 3–7 (OpenAI, 2026a). These figures were produced from the implemented architecture and recorded experimental data, rather than generated as raster illustrations. The tools also assisted with proposed experimental classifications and semantic reviews, as described in Section 6; automated proposals remain distinguished from explicit acceptance decisions. The authors take responsibility for the manuscript, its analysis, and the reported results.

[8] AI-assisted writing and translation used OpenAI tools (OpenAI, 2026a); see the disclosure at the end of the paper.

REFERENCES

Besold, T. R., d'Avila Garcez, A., Bader, S., Bowman, H., Domingos, P., Hitzler, P., Kuehnberger, K.-U., Lamb, L. C., Lowd, D., Lima, P. M. V., de Penning, L., Pinkas, G., Poon, H., and Zaverucha, G. (2017). Neural-symbolic learning and reasoning: A survey and interpretation. *arXiv preprint arXiv:1711.03902*.

d'Avila Garcez, A., Besold, T. R., De Raedt, L., Földiák, P., Hitzler, P., Icard, T., Kühnberger, K.-U., Lamb, L. C., Miikkulainen, R., and Silver, D. L. (2015). Neural-symbolic learning and reasoning: Contributions and challenges. In *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches—Papers from the 2015 AAAI Spring Symposium*, pages 18–21. AAAI Press.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115.

Giarratano, J. C. and Riley, G. D. (2005). *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th edition.

Hart, J., Moriarty, A., Pasternak, K., Kummert, J., Leonetti, M., Contreras, L., Ruegemer, L., Mitzutani, A., Ribeiro, T., Golding, A., Kang, T., and Pimentel, F. (2026). RoboCup@Home 2026: Rules and regulations. RoboCup@Home Technical Com-

mittee. https://github.com/RoboCupAtHome/RuleBook/releases/tag/2026.2.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., and Stone, P. (2023). LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.

NASA Johnson Space Center (1991). *CLIPS Reference Manual*. NASA.

OpenAI (2026a). ChatGPT and Codex: tools used for manuscript and code assistance. https://openai.com/. Accessed October 2026. Tool use is described in the manuscript disclosure.

OpenAI (2026b). OpenAI developer documentation and model guides. https://developers.openai.com/.

OpenAI (2026c). OpenAI platform usage dashboard. https://platform.openai.com/usage.

RoboCup@Home (2026a). CommandGenerator: Command generator for RoboCup@Home. https://github.com/RoboCupAtHome/CommandGenerator.

RoboCup@Home (2026b). CompetitionTemplate. https://github.com/RoboCupAtHome/CompetitionTemplate.

Savage, J. et al. (2024). ViRBOT: A modular architecture for service robots with hybrid planning. In *RoboCup Symposium*.

Shi, L., Chen, X., Hu, Y., Li, J., Zhang, H., and Zhao, H. (2023). ChatGPT for robotics: Design principles and model abilities. *Microsoft Research Technical Report*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.